Module: `Problem 3 - Using Bisection Search to Make the Program Faster`

> This module provides a helper function that simulates the evolution of a credit-card balance over the remaining months of a year when a fixed minimum payment is applied each month. It is intended to be used inside a bisection-search routine that determines the smallest fixed payment needed to pay off the balance within 12 months.

Functions

`calculate(month, balance, minPay, monthlyInterestRate)`

Description

Computes the credit-card balance after repeatedly applying a fixed minimum payment (`minPay`) and accruing interest (`monthlyInterestRate`) for each month until the end of a 12-month period. The function starts counting from the supplied `month` value, updates the balance month-by-month, and returns the final balance after the loop completes.

Parameters - `month` (*int*): The starting month index (0-based). The loop runs while `month < 12` . - `balance` (*float*): The current outstanding balance at the start of the calculation. - `minPay` (*float*): The fixed minimum payment to be subtracted from the balance each month. - `monthlyInterestRate` (*float*): The monthly interest rate expressed as a decimal (e.g., `0.01875` for 1.875 % per month).

Returns

`float` : The balance remaining after processing months up to (and including) month 11. If `month` is already `≥ 12` , the original `balance` is returned unchanged.

Examples

```
# Example: simulate balance from month 0 with a $400 balance,
# a $50 fixed payment, and a 2 % monthly interest rate.
final_balance = calculate(
    month=0,
    balance=400.0,
    minPay=50.0,
    monthlyInterestRate=0.02
)
# → final_balance ≈ 232.68
```

(The numeric result above is derived from the function's logic; actual values may differ based on input.)

Edge Cases / **Notes** - If `month` is already 12 or greater, the `while` loop never executes and the function instantly returns the input `balance` . - A negative `minPay` would *increase* the balance each iteration, which is typically unintended. - Passing a negative `monthlyInterestRate` will reduce the balance faster than intended; the function does not guard against such values. - The function does **not** perform any rounding; the returned balance retains full floating-point precision.